

Thuật Toán Binary Search (Tìm kiếm nhị phân)

Hoàng-Nguyên Vũ

1. Khái niệm

Binary Search là thuật toán tìm kiếm nhị phân, giúp tìm kiếm phần tử trong mảng đã sắp xếp một cách hiệu quả. Thuật toán hoạt động bằng cách chia đôi khoảng tìm kiếm trong mỗi bước so sánh.

2. Điều kiện áp dụng

- Mảng phải được **sắp xếp**.
- Có thể triển khai bằng **vòng lặp** hoặc **đệ quy**.

3. Độ phức tạp

- Trường hợp tốt nhất: $O(1)$
- Trung bình và xấu nhất: $O(\log n)$
- Không gian: $O(1)$ nếu dùng vòng lặp, $O(\log n)$ nếu đệ quy

4. Giải mã

```
1 1. Đặt left = 0, right = n - 1
2 2. Trong khi left <= right:
3   a. mid = (left + right) // 2
4   b. Nếu arr[mid] == x      trả về mid
5   c. Nếu arr[mid] < x      tìm bên phải (left = mid + 1)
6   d. Nếu arr[mid] > x      tìm bên trái (right = mid - 1)
7 3. Nếu không tìm thấy    trả về -1
```

5. Cài đặt Python (vòng lặp)

```
1 def binary_search(arr, x):
2     left = 0
3     right = len(arr) - 1
4     while left <= right:
5         mid = (left + right) // 2
6         if arr[mid] == x:
7             return mid
```

```
8         elif arr[mid] < x:
9             left = mid + 1
10        else:
11            right = mid - 1
12        return -1
```

6. Bài tập thực hành

Bài 1: Tìm vị trí phần tử

Viết hàm `binary_search(arr, x)` trả về chỉ số của `x` nếu có, ngược lại trả về `-1`.

Ví dụ:

- `arr = [1, 3, 5, 7, 9]`, `x = 5` → Output: 2
- `x = 2` → Output: -1

Bài 2: Tìm phần tử đầu tiên xuất hiện

Viết hàm tìm chỉ số đầu tiên của phần tử `x` trong mảng có thể chứa nhiều phần tử giống nhau.

Ví dụ: `arr = [1, 2, 4, 4, 4, 5]`, `x = 4` → Output: 2

Bài 3: Tìm phần tử nhỏ nhất $\geq x$

Trả về chỉ số của phần tử đầu tiên trong mảng lớn hơn hoặc bằng `x`.

Ví dụ:

- `arr = [1, 3, 5, 7, 9]`, `x = 6` → Output: 3
- `x = 10` → Output: -1

Bài 4: Đếm số lần gọi đệ quy

Viết lại Binary Search dưới dạng đệ quy, và trả về số lần gọi hàm tìm kiếm.